Fatima Jinnah Women University



Department of Computer Science
DevOps Lab # 6

Name: Hina Ali
Registration no: 2023-BCS-040
Course code: CS-649
Section: BCS VII(B)
GitHub Username: Hina-Ali32

Getting Started

Task 1: Run Nexus Repository and Complete the Initial Setup

```
hina-ali32@ubuntu:~$ docker ps | grep nexus
b7341fd3b9f8   sonatype/nexus3:latest   "/opt/sonatype/nexus…"   9 minutes ago   Up 9 minutes   0.0.0.0:8007->8007/tcp,
[::]:8007->8007/tcp, 0.0.0.0:8006->8081/tcp, [::]:8006->8081/tcp   nexus
hina-ali32@ubuntu:~$
```

It shows the Nexus container running successfully in Docker with the required ports configured.

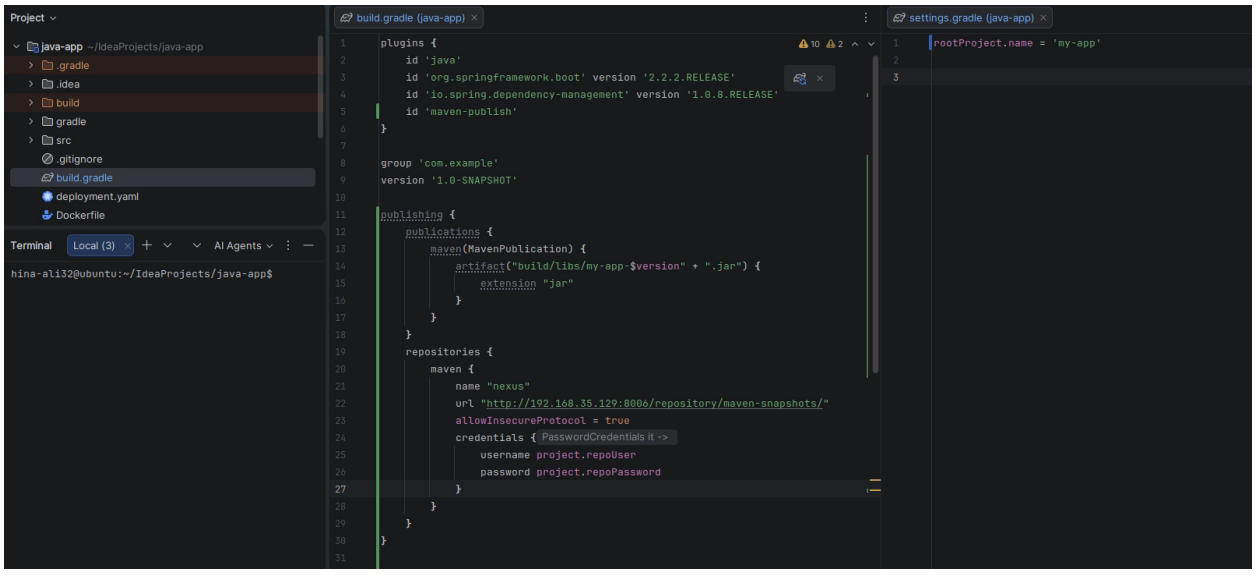
Task 2: Create a Role and User and Verify Access

```
hina-ali32@ubuntu:~$ curl -i -u nexus http://192.168.35.129:8006/service/rest/v1/status
Enter host password for user 'nexus':
HTTP/1.1 200 OK
Server: Nexus/3.98.1-01 (COMMUNITY)
Vary: Accept-Encoding
X-Content-Type-Options: nosniff
Content-Security-Policy: default-src http: data: blob: 'unsafe-inline'; script-src http: 'unsafe-inline' 'unsafe-eval'
Content-Length: 0

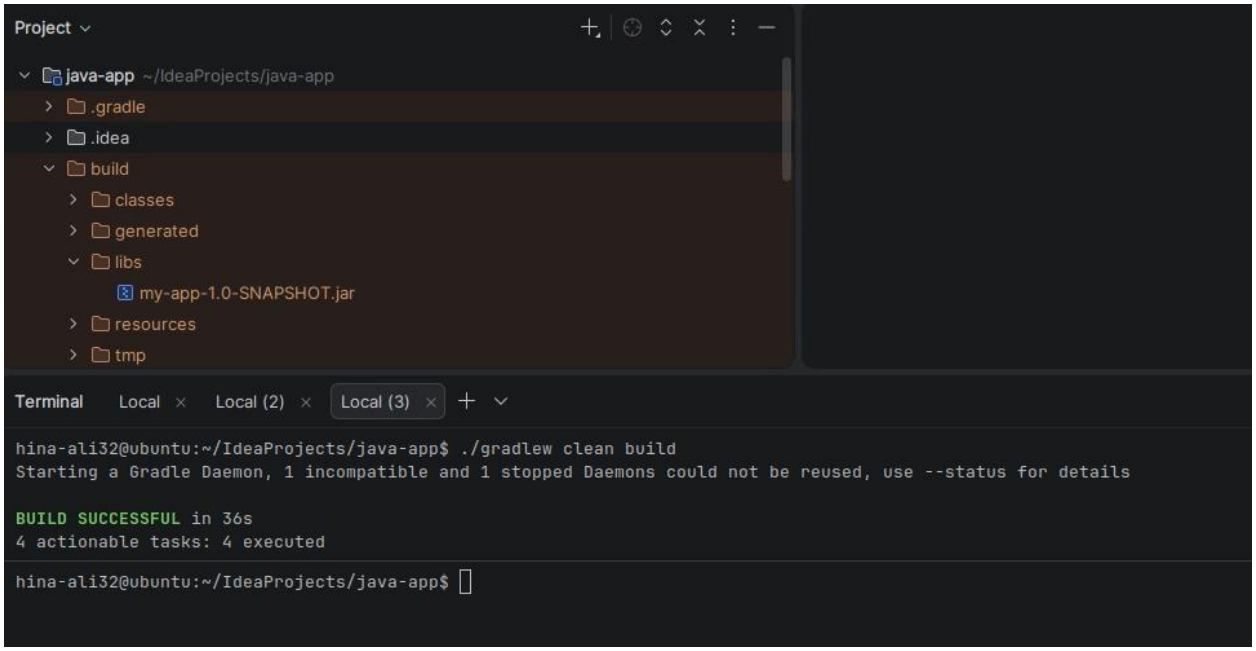
hina-ali32@ubuntu:~$
```

It shows successful authentication of the local Nexus user with an HTTP 200 OK response.

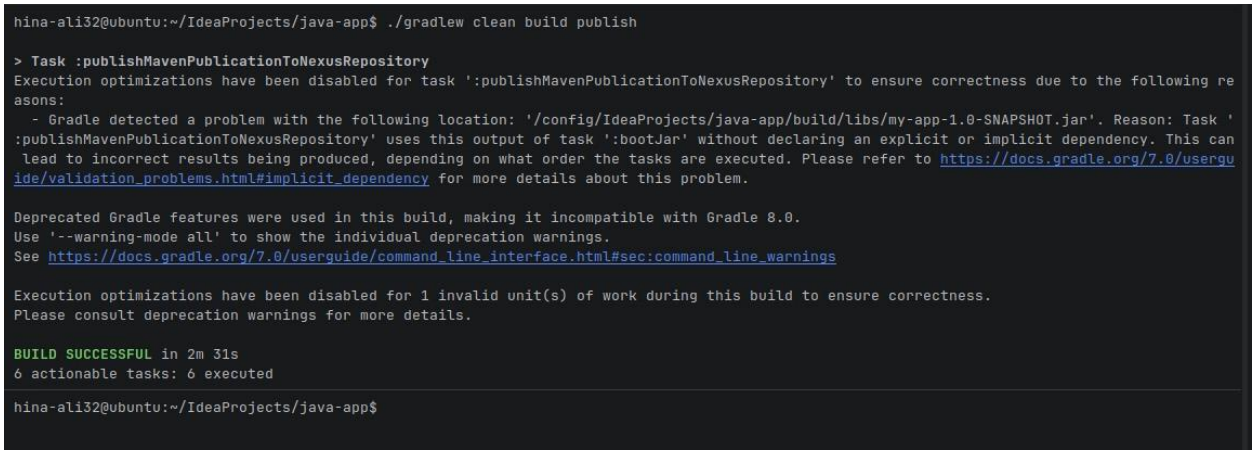
Task 3: Build and Publish the Gradle Application to Nexus



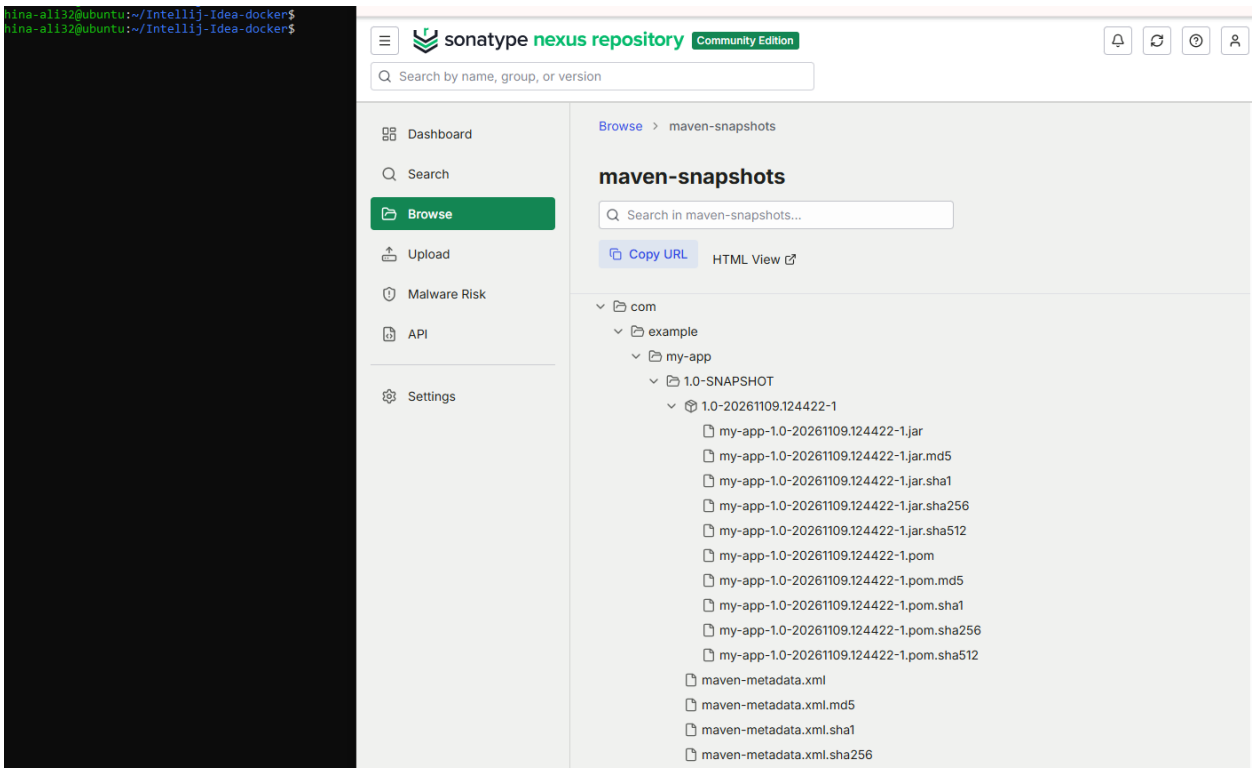
It shows the Gradle project configuration for publishing the application to the Nexus repository.



It shows the successful Gradle build and generation of the application JAR file.

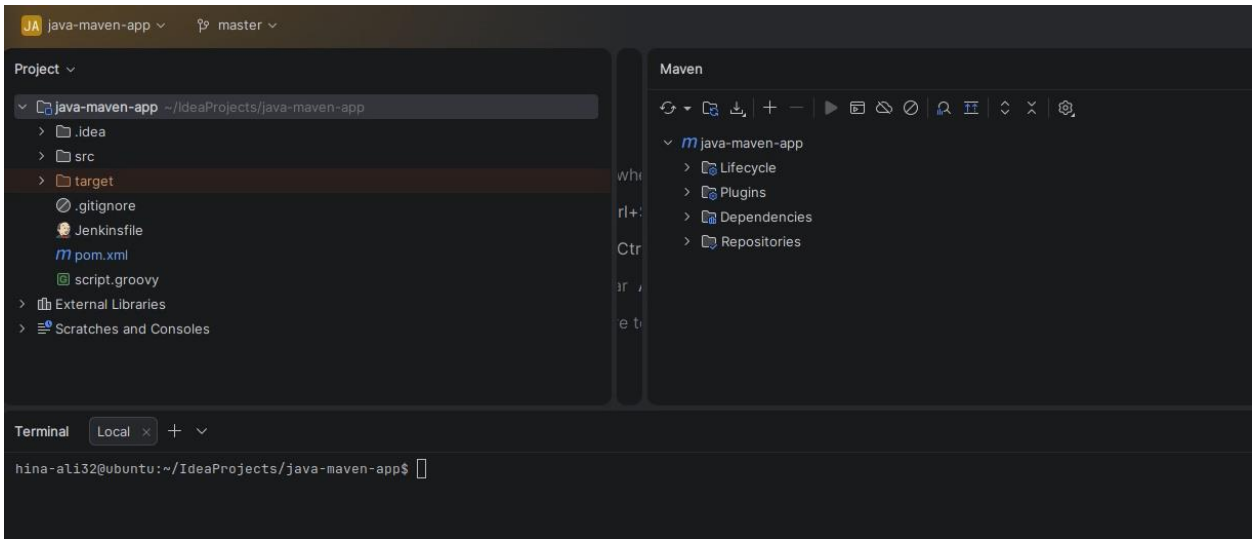


Shows the Gradle application being published to Nexus. The ./gradlew clean build publish command built the project and uploaded the generated JAR file to the Maven snapshots repository. The output displays the successful publication.

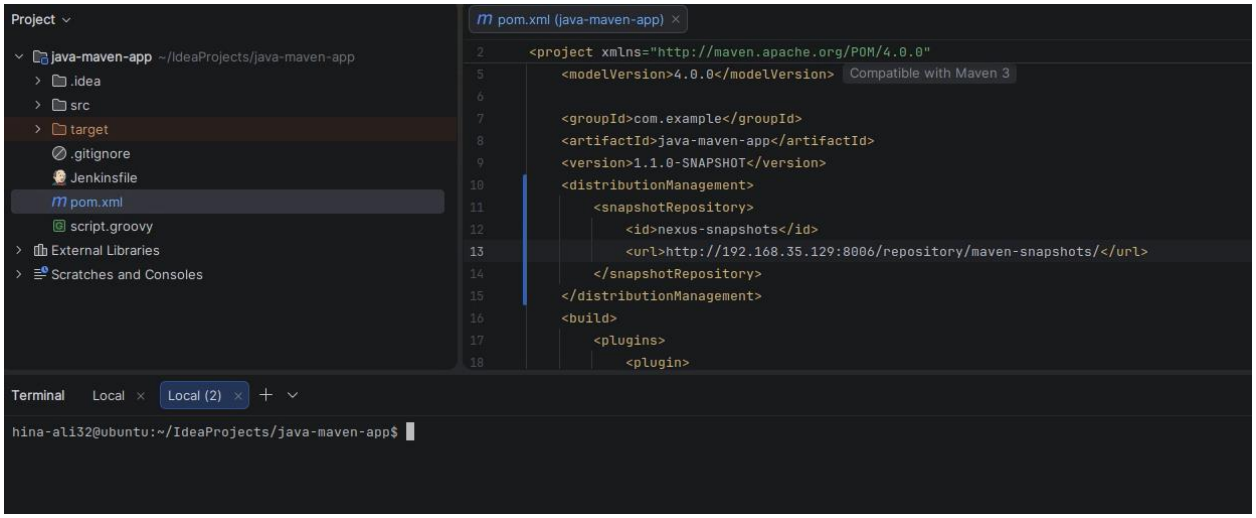


Shows the published Gradle application inside the Nexus Maven snapshots repository. The com/example/my-app/1.0-SNAPSHOT directory contains the generated JAR file, POM file, and related Maven metadata.

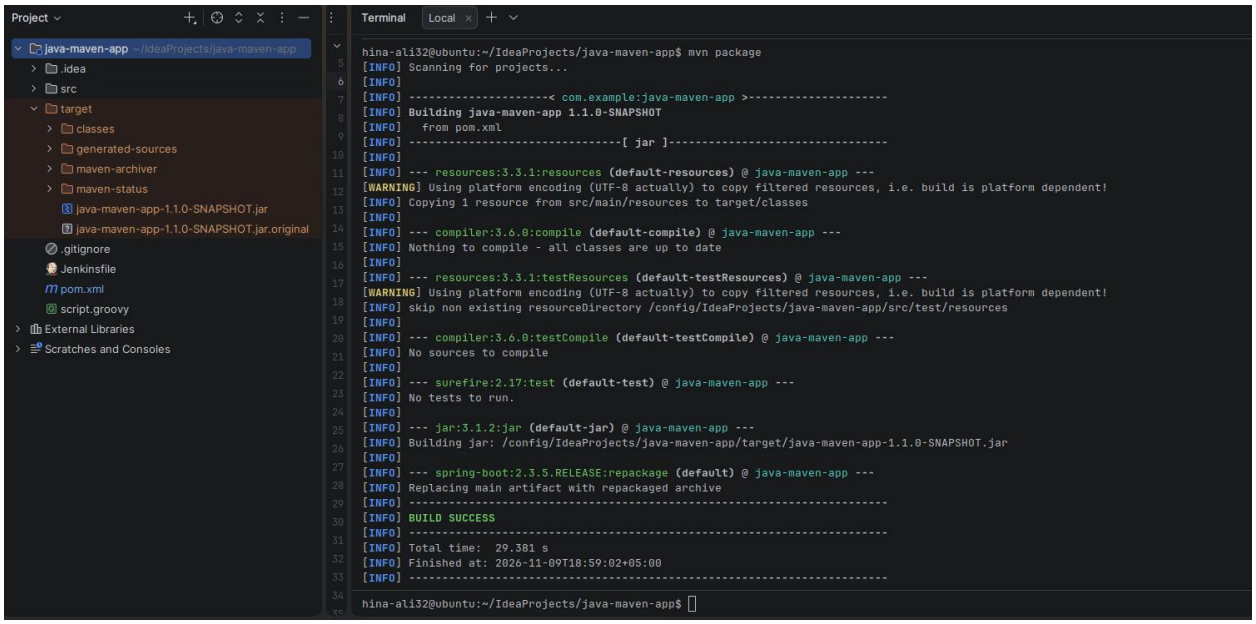
Task 4: Publish the Maven Application to Nexus



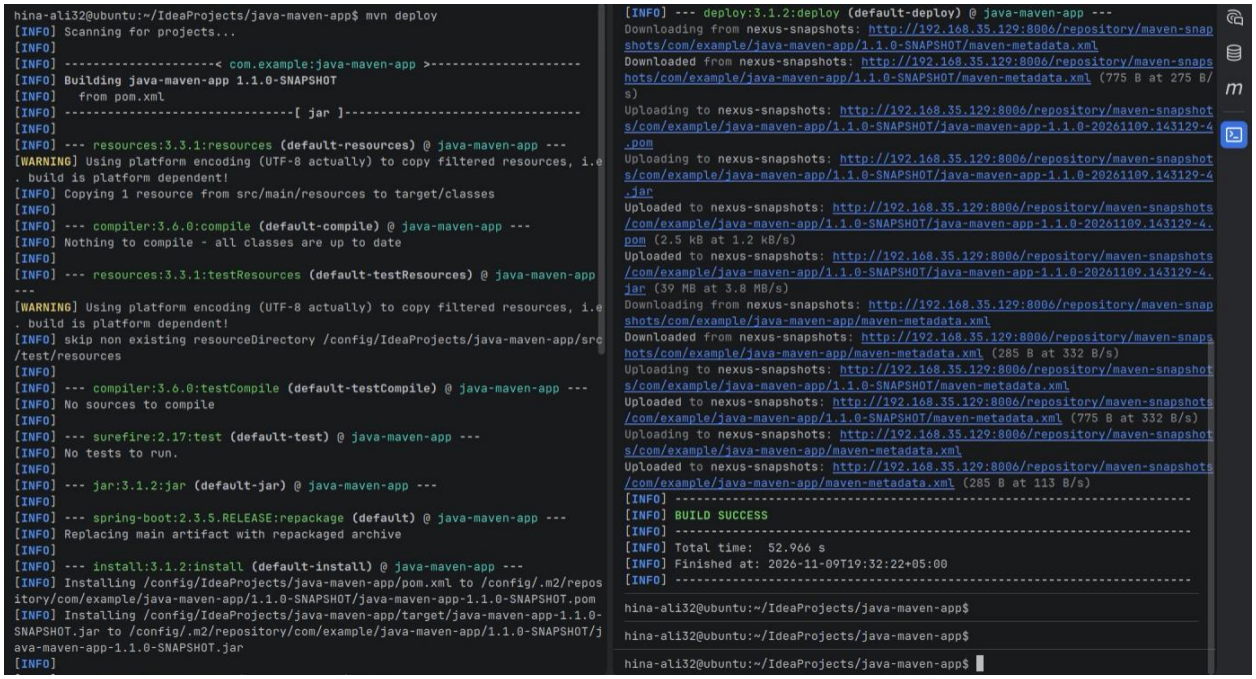
Shows the java-maven-app project cloned from GitLab and opened in IntelliJ IDEA. The repository was downloaded using the provided GitLab URL and opened for Maven configuration.



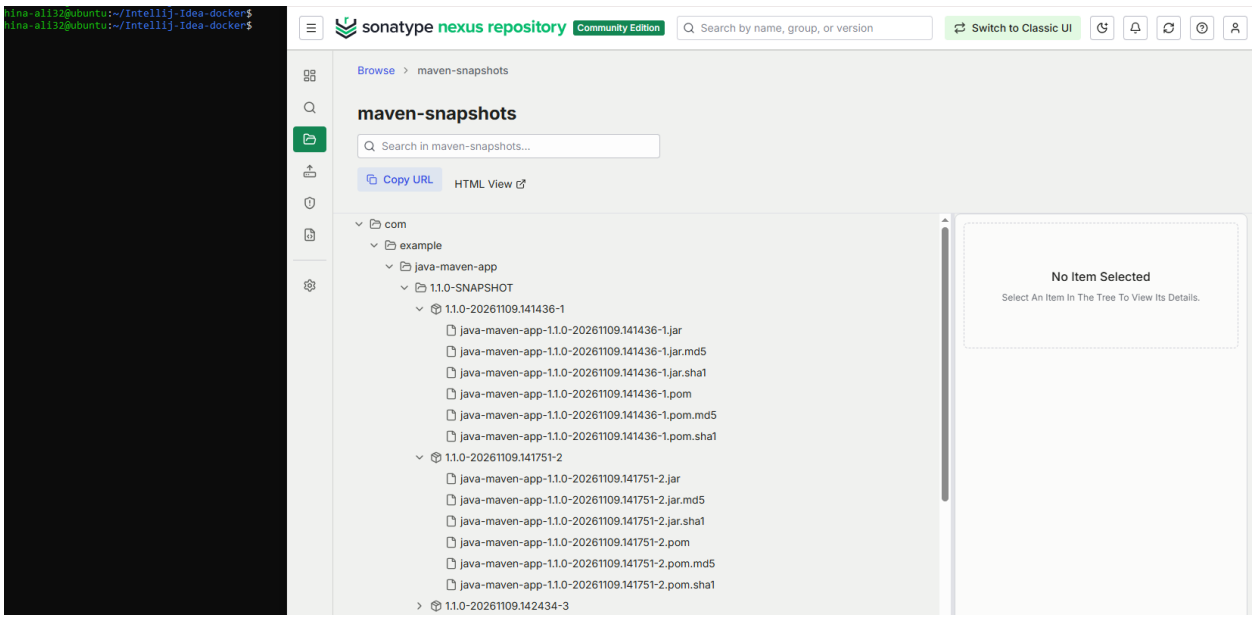
Shows the Maven project configured to publish artifacts to Nexus. The distributionManagement block was added to pom.xml with the repository ID nexus-snapshots and the Nexus Maven snapshots URL.



Shows the Maven application being packaged in IntelliJ IDEA. The mvn package command compiled the project and generated the snapshot JAR file inside the target directory. The output displays BUILD SUCCESS.

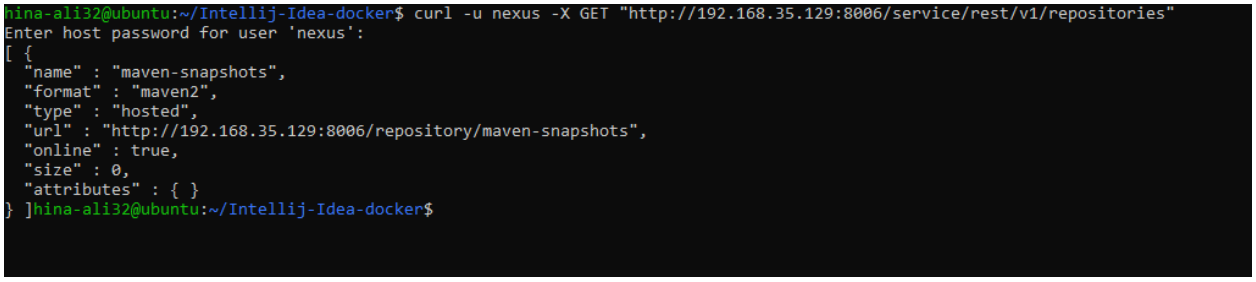


Shows the Maven application being deployed to Nexus. The mvn deploy command packaged the application and uploaded the JAR, POM, and related metadata to the Maven snapshots repository. The output displays BUILD SUCCESS.

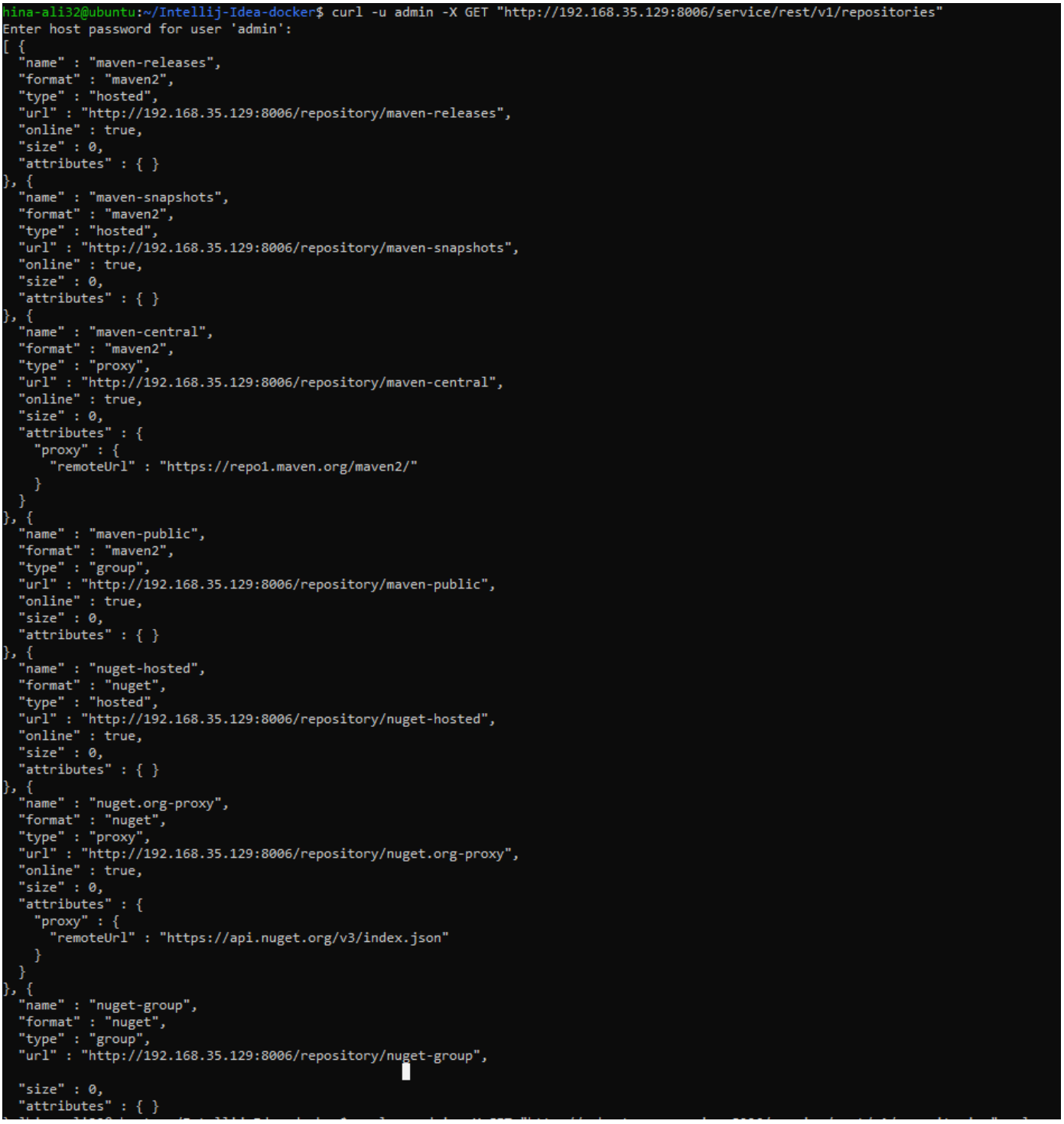


Shows the published Maven application inside the Nexus repository. The com/example/java-maven-app/1.1.0-SNAPSHOT directory contains the published JAR file, POM file, and Maven metadata.

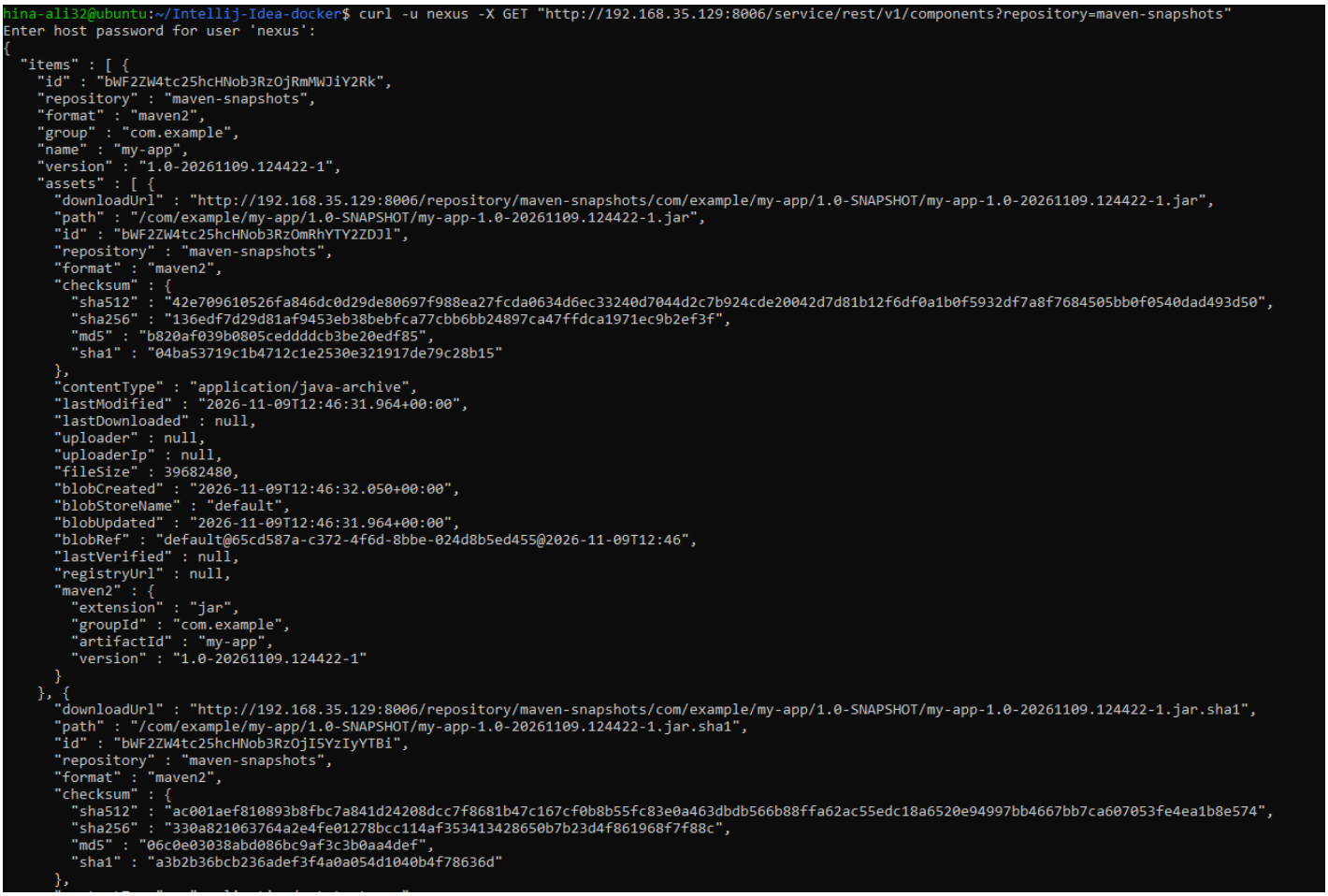
Task 5: Inspect Nexus Repositories and Components Using the REST API



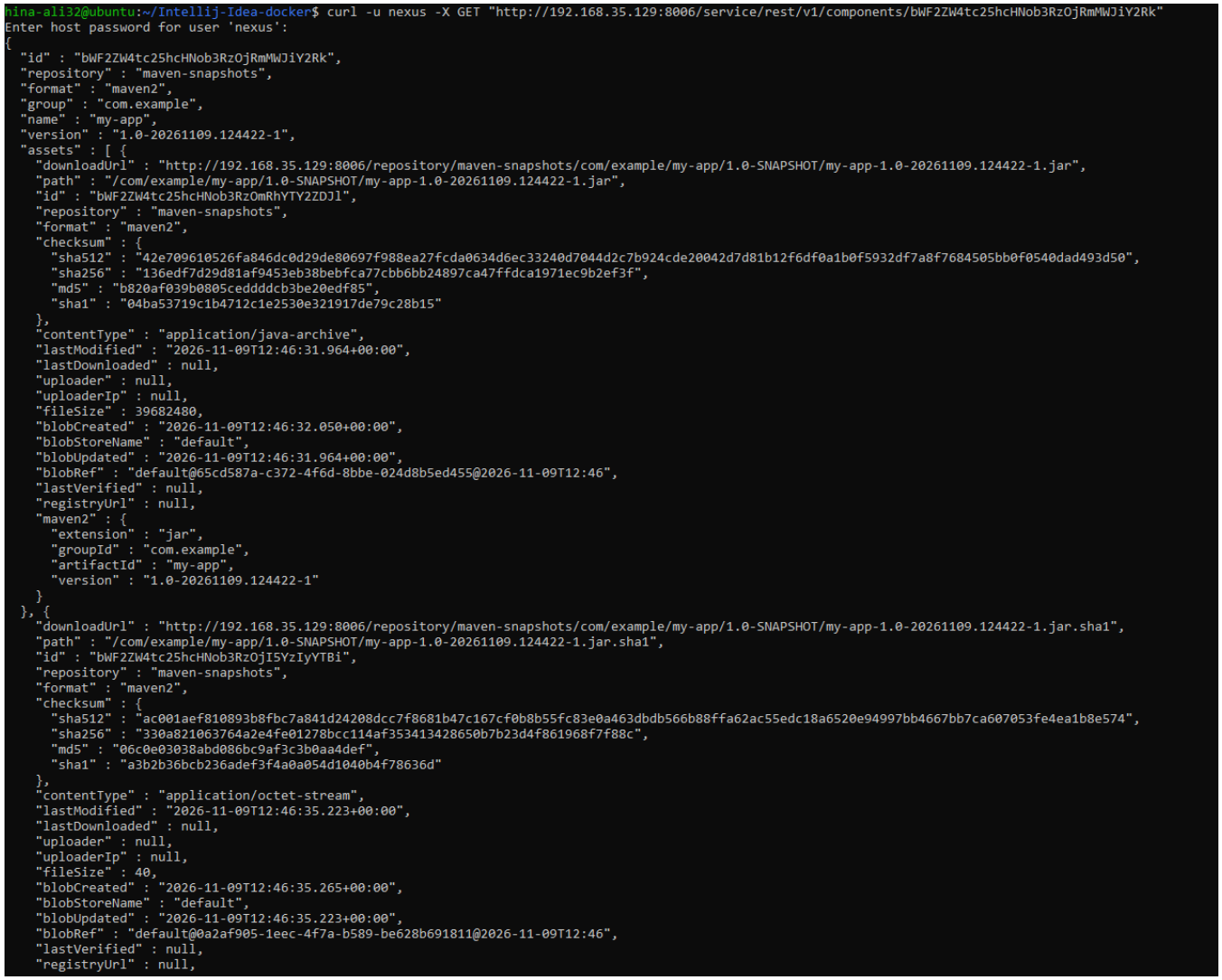
Shows the repositories requested using the restricted Nexus user. The curl -u nexus command sent a GET request to the Nexus REST API endpoint /service/rest/v1/repositories and returned the accessible repository information in JSON format.



Shows the repositories requested using the Nexus administrator account. The curl -u admin command sent a GET request to the same REST API endpoint and returned the repository information in JSON format.



Shows the components available in the Maven snapshots repository. The curl -u nexus command requested /service/rest/v1/components?repository=maven-snapshots and returned component information, including IDs, names, and versions.

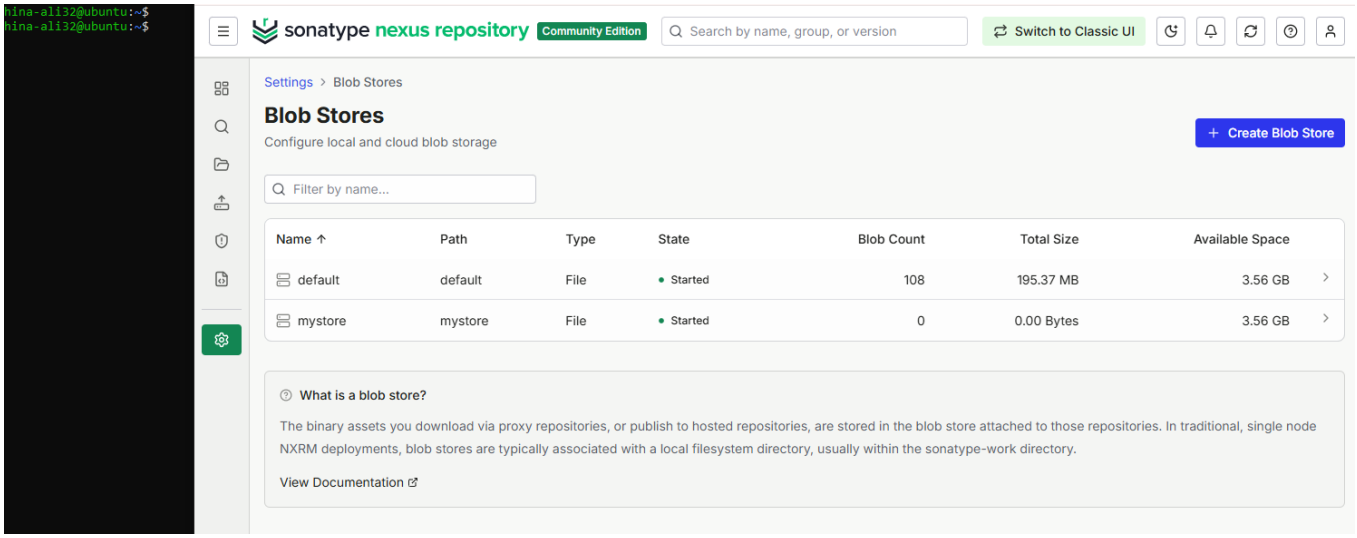


Shows the details of a specific Nexus component. The curl -u nexus command used the selected component ID with the REST API endpoint /service/rest/v1/components/ to retrieve its repository, format, group, name, version, and asset information.

Task 6: Create a Nexus Blob Store and Docker Hosted Repository

```
hina-ali32@ubuntu:~/Intellij-Idea-docker$ docker exec -it nexus /bin/sh
/opt/sonatype $ pwd
/opt/sonatype
/opt/sonatype $ cd sonatype-work/nexus3
/opt/sonatype/sonatype-work/nexus3 $ cd blobs
/opt/sonatype/sonatype-work/nexus3/blobs $ ls
default
/opt/sonatype/sonatype-work/nexus3/blobs $ ls default/content
2026        directpath  tmp
/opt/sonatype/sonatype-work/nexus3/blobs $
```

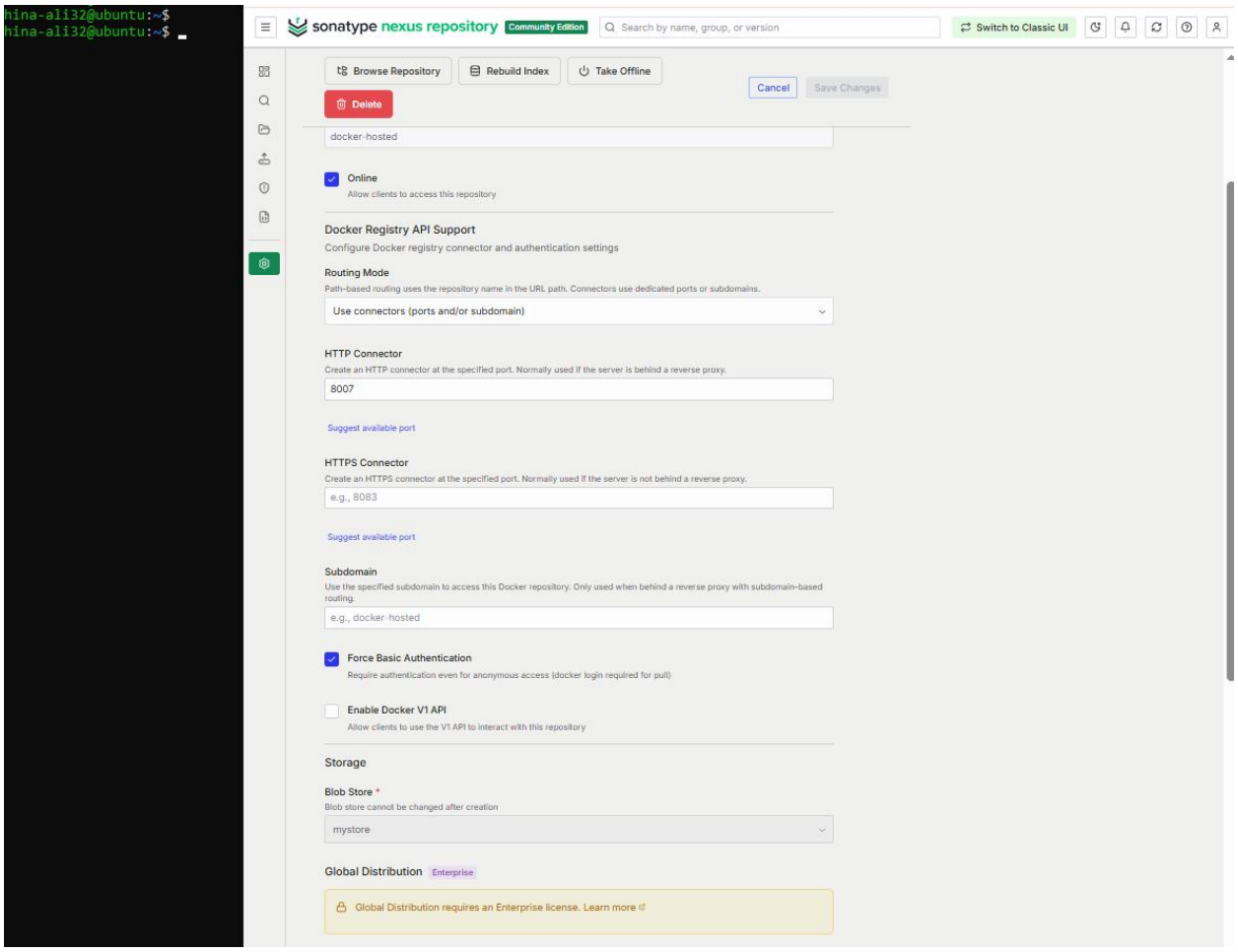
Shows the default blob store directory inside the Nexus container. The docker exec -it nexus /bin/sh command opened the container shell, and the cd and ls commands displayed the blob storage directory and its contents.



Shows a new file blob store named mystore created in Nexus. The blob store was configured with the path /nexus-data/blobs/mystore, while the existing default blob store remained available.

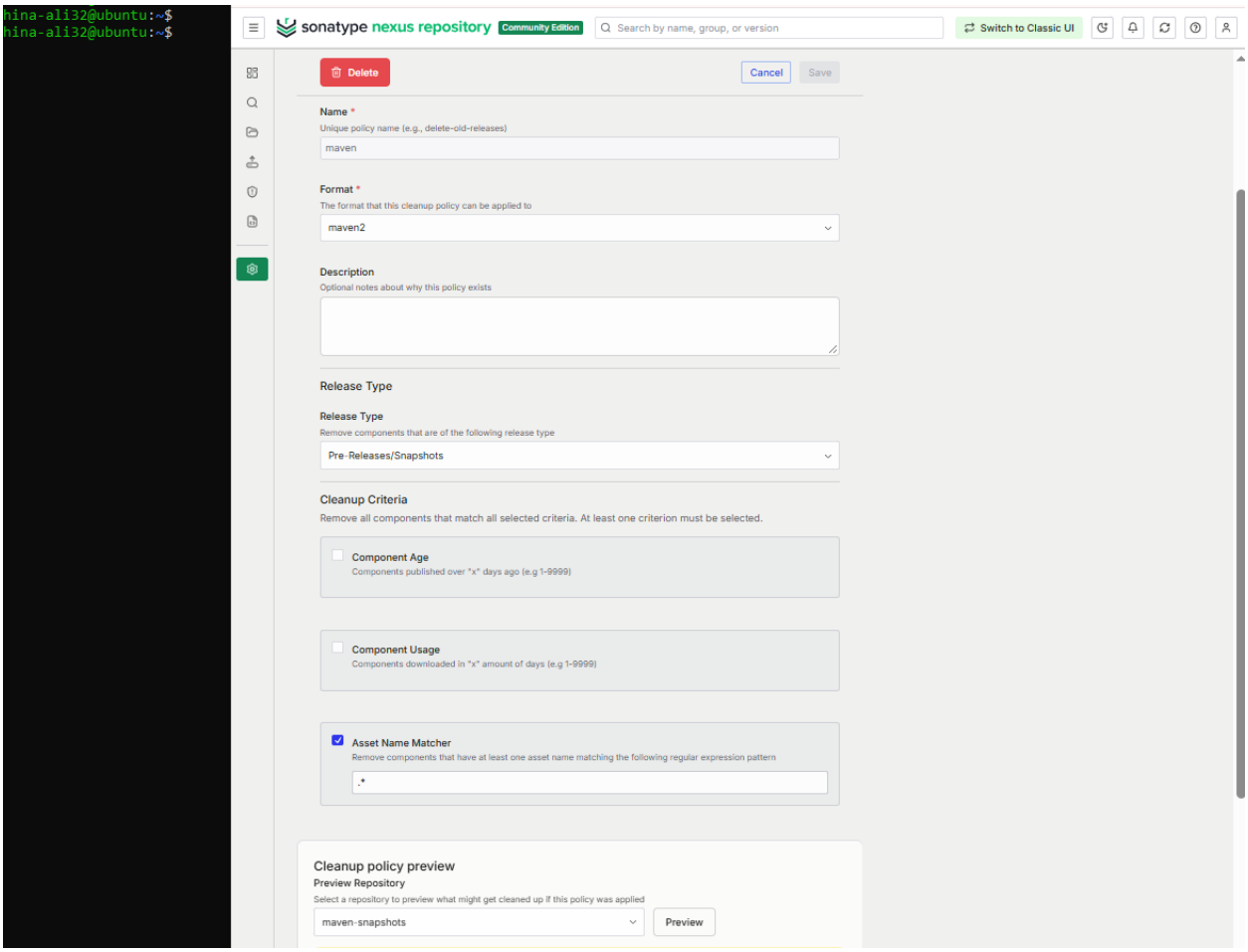
```
hina-ali32@ubuntu:~/Intellij-Idea-docker$ docker exec -it nexus /bin/sh
/opt/sonatype $ pwd
/opt/sonatype
/opt/sonatype $ cd sonatype-work/nexus3
/opt/sonatype/sonatype-work/nexus3 $ cd blobs
/opt/sonatype/sonatype-work/nexus3/blobs $ ls
default
/opt/sonatype/sonatype-work/nexus3/blobs $ ls default/content
2026        directpath  tmp
/opt/sonatype/sonatype-work/nexus3/blobs $ pwd
/opt/sonatype/sonatype-work/nexus3/blobs
/opt/sonatype/sonatype-work/nexus3/blobs $ ls
default  mystore
/opt/sonatype/sonatype-work/nexus3/blobs $ ls mystore
content             metadata.properties
/opt/sonatype/sonatype-work/nexus3/blobs $ exit
hina-ali32@ubuntu:~/Intellij-Idea-docker$
```

Shows the default and mystore directories inside the Nexus container. The pwd and ls commands displayed both directories, and the exit command returned to the Ubuntu server terminal.

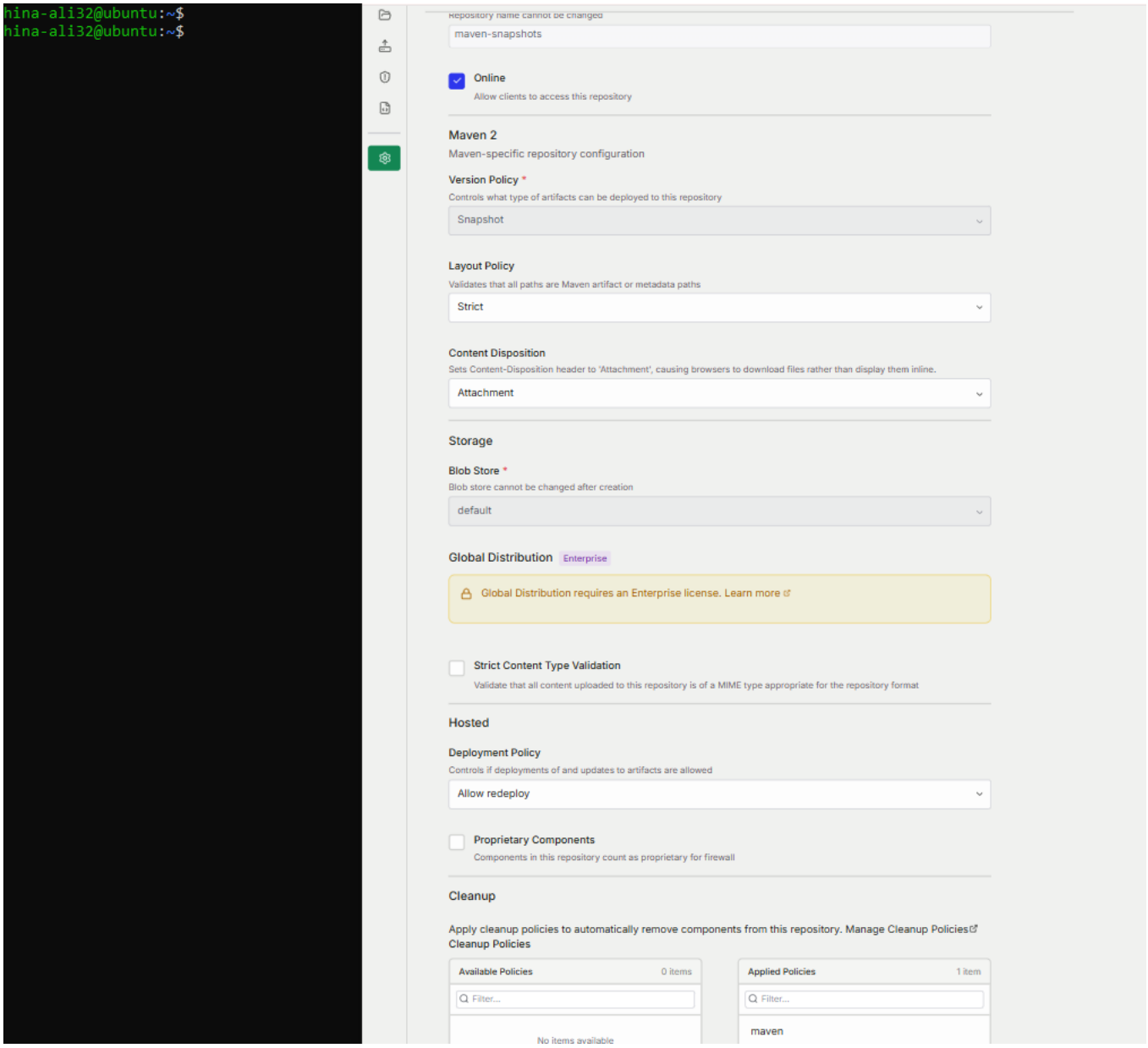


Shows the Docker hosted repository created in Nexus. The repository was named docker-hosted and configured with HTTP connector port 8007, disabled anonymous pulls, and the mystore blob store.

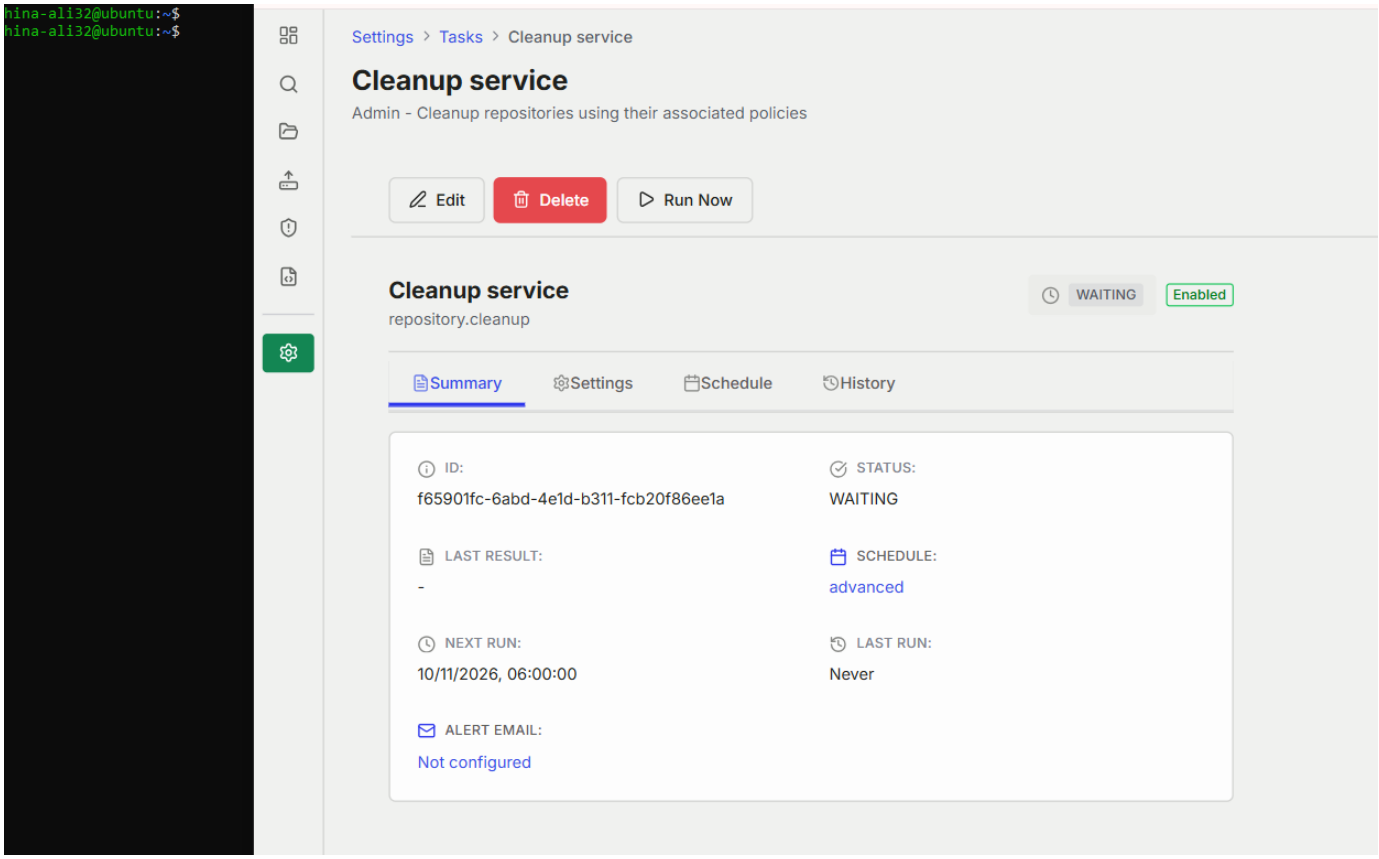
Task 7: Create and Run a Maven Snapshot Cleanup Policy



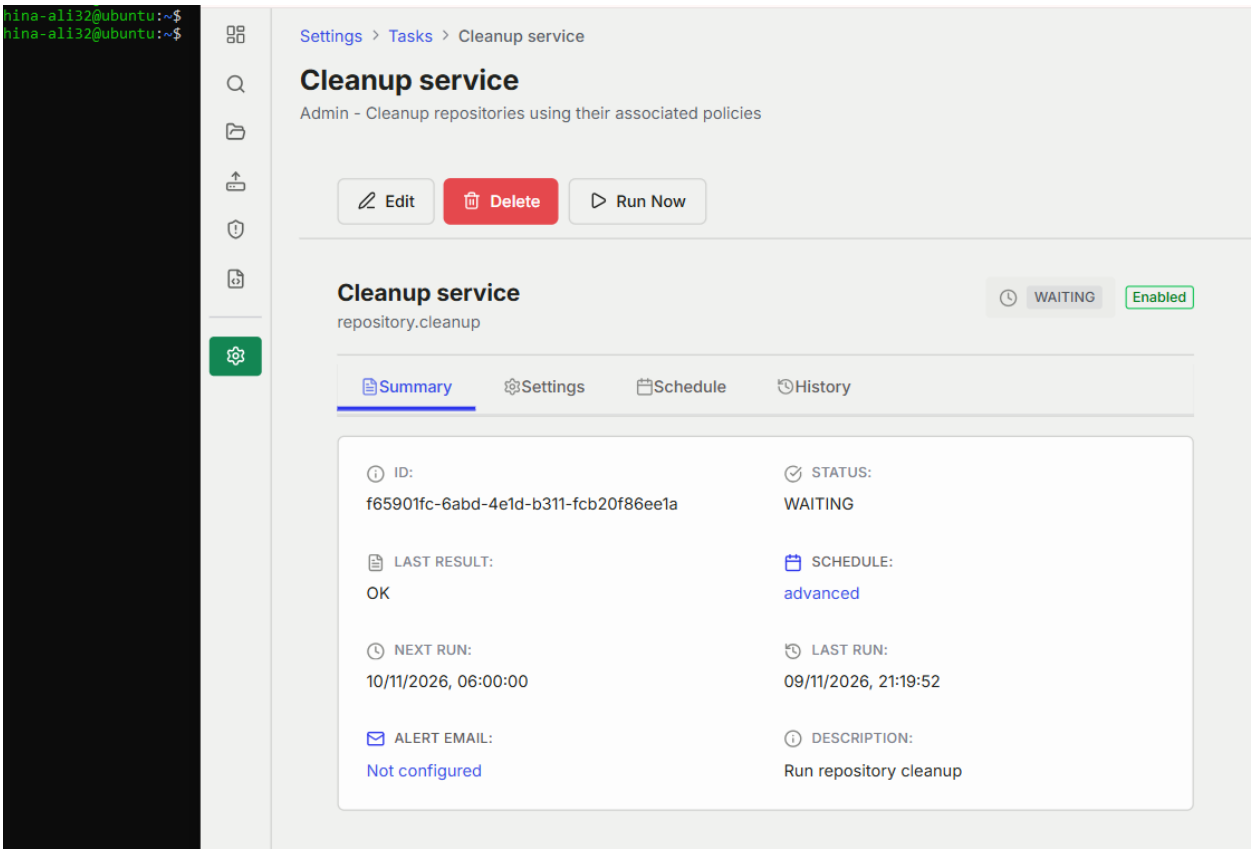
Shows a Maven cleanup policy created in Nexus. The policy was configured for Maven snapshots using the .* regular expression to match all snapshot assets. The preview displays the matching components from the maven-snapshots repository.



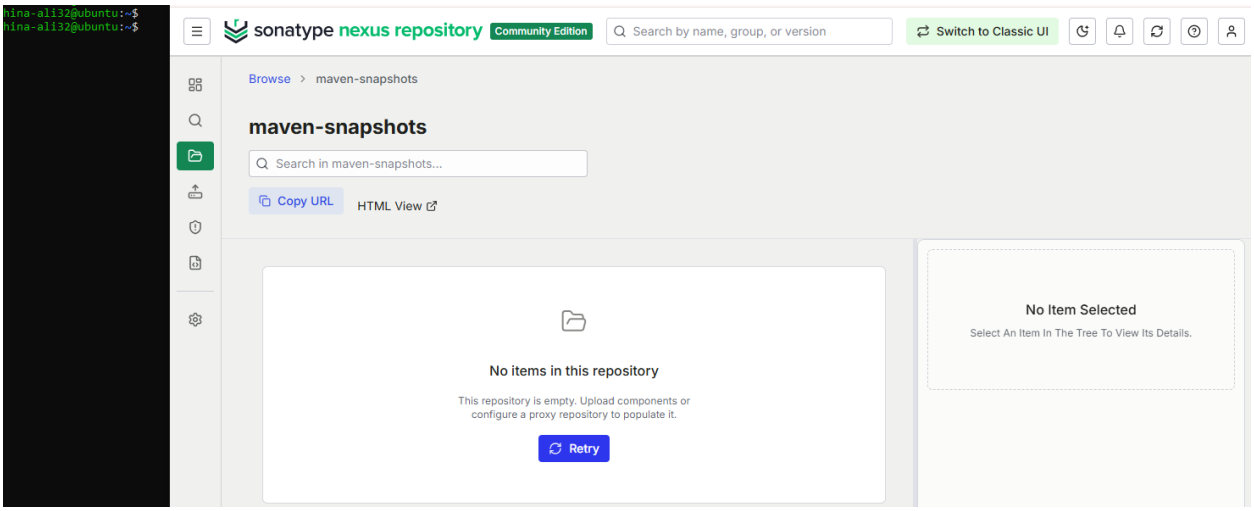
Shows the Maven cleanup policy attached to the maven-snapshots repository. The maven policy was moved from Available to Applied and saved in the repository settings.



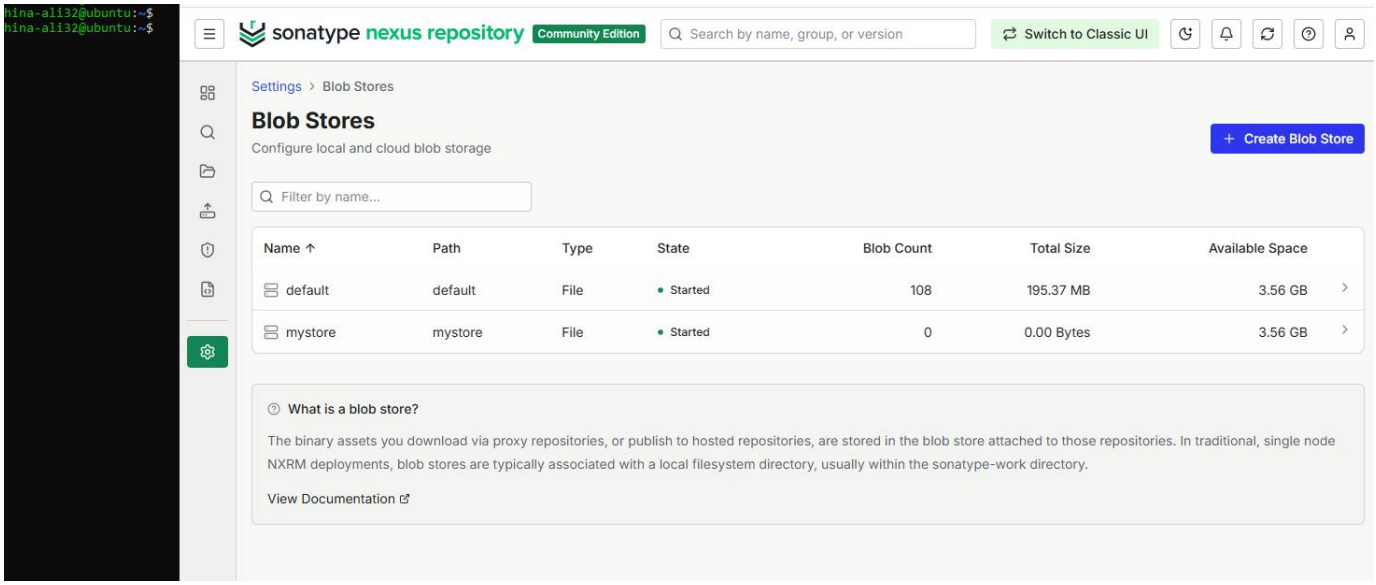
Shows the Cleanup service configured for daily execution at 06:00 AM. The task status is Waiting, indicating that it is scheduled for its next execution.



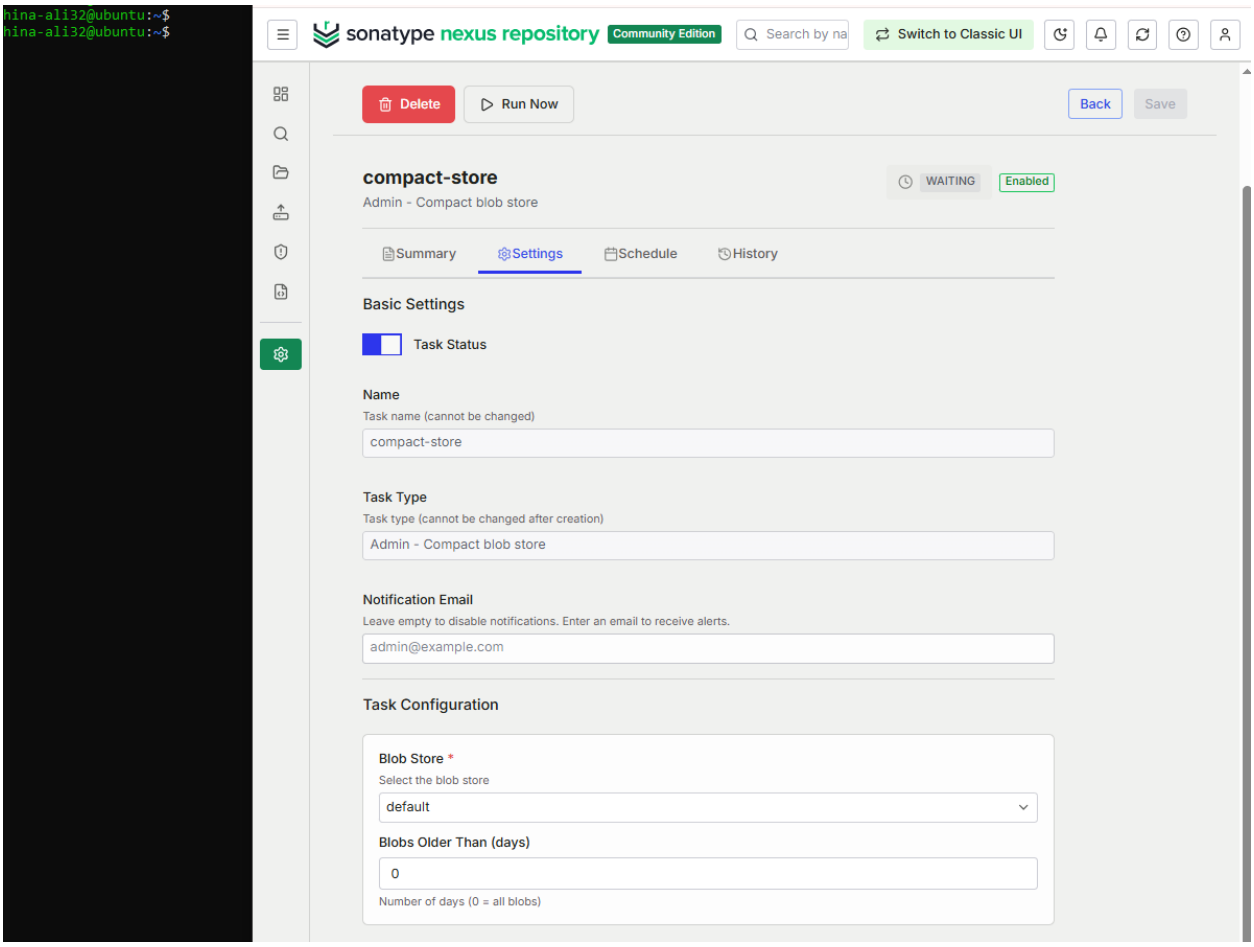
Shows the Cleanup service after manual execution. The Run option was selected to execute the cleanup task, and the task status returned to Waiting with Last result marked as OK.



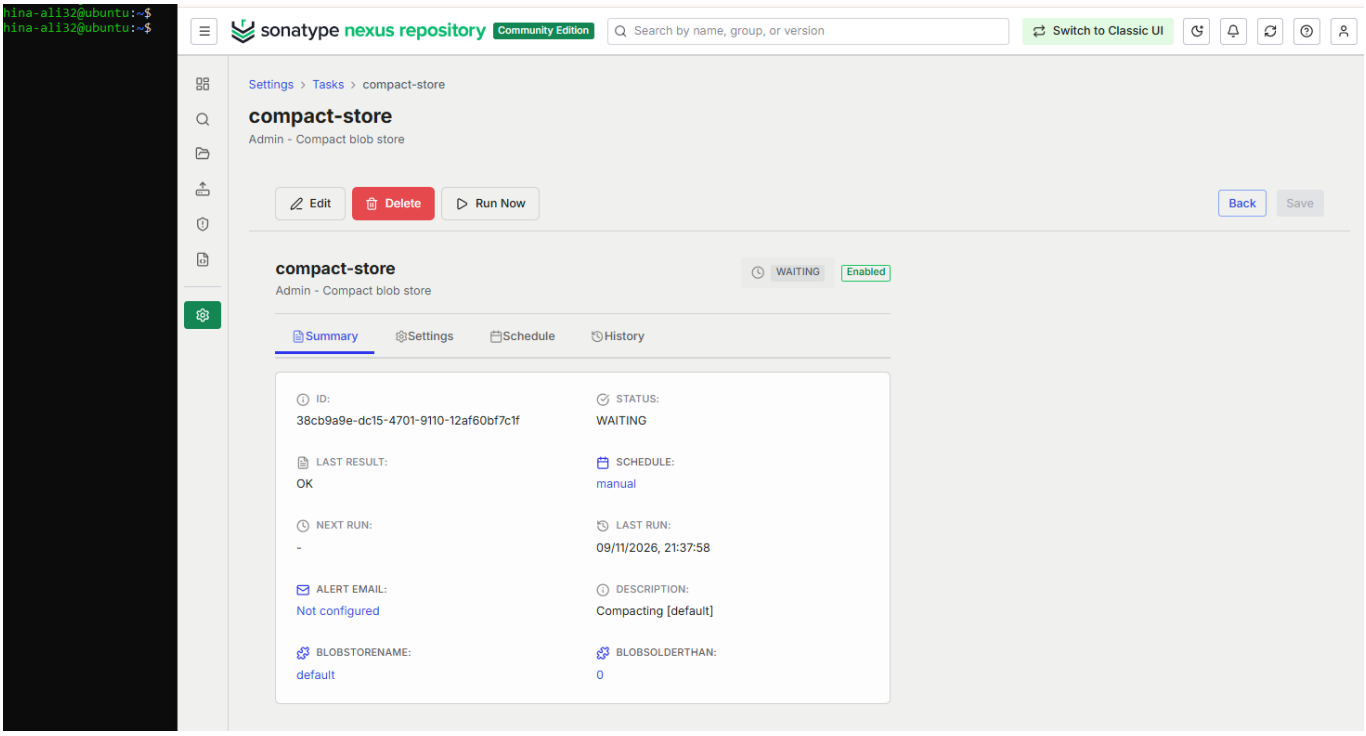
Shows the maven-snapshots repository after the cleanup process. The cleanup service removed the matching snapshot components, and the repository displays the message No component/assets found in repository.



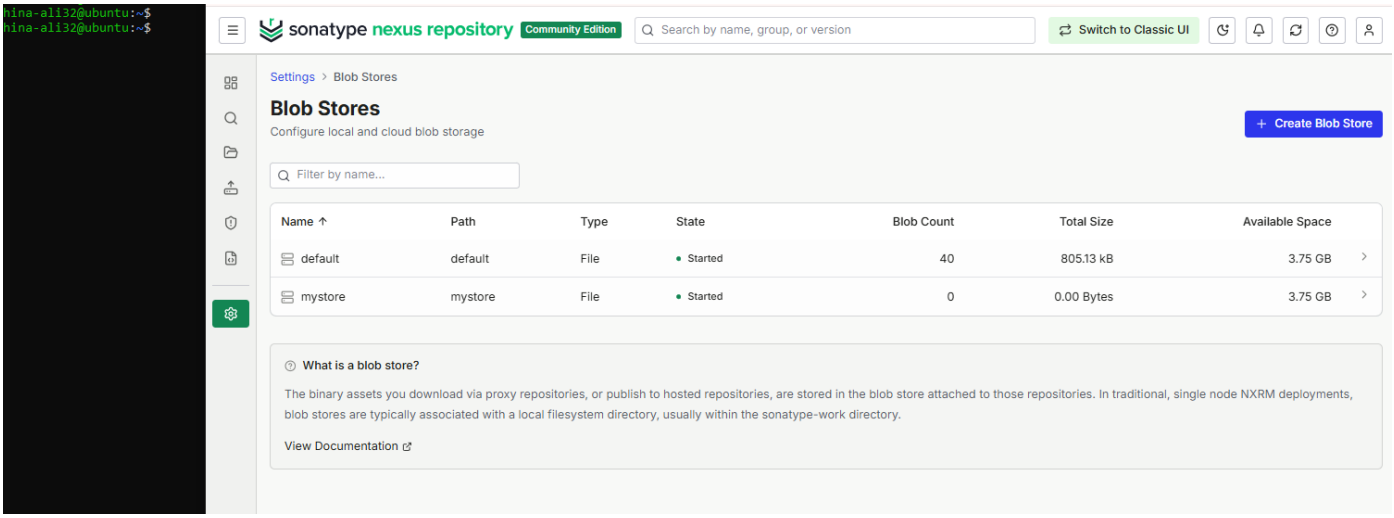
Shows the default blob store storage statistics before compaction. The Blob Count, Used Size, and Available Space values were recorded to compare storage usage before and after the compaction process.



Shows the compact-store task configured in Nexus. The task was named compact-store, assigned to the default blob store, and scheduled to run daily at 06:30 AM with the Blobs Older Than setting set to 0 days.



Shows the compact-store task after manual execution. The Run option was selected to permanently remove eligible soft-deleted blobs, and the task completed with Last result marked as OK.



Shows the default blob store storage statistics after compaction. The updated Blob Count, Used Size, and Available Space values demonstrate the storage space reclaimed by the compaction process.